Assignment: Bank Account Management System using C++

Problem Statement

Create a class named BankAccount to represent a bank account. Each account should store the account holder's name and current balance. The program must allow users to perform basic banking operations such as depositing money, withdrawing money, and applying interest to the account balance.

The system should also maintain information that is common to all bank accounts, such as the total number of accounts created and a shared interest rate, using static data members and static member functions.

Requirements

1. Use a class to model a bank account.
2. Use private data members to store:
    - Account holder name
    - Account balance
3. Implement a constructor to initialize account details and increment the total number of accounts.
4. Provide member functions to:
    - Deposit a valid amount into the account
    - Withdraw an amount with proper validation
    - Apply interest to the account balance
    - Display account details in a formatted manner
5. Use static data members to:
    - Store a common interest rate for all accounts
    - Keep track of the total number of bank accounts created
    - Keep track of the total money / balance in the bank
6. Use static member functions to:
    - Display the total number of accounts
    - Display the total balance in the bank

Demonstrate the working of the program by creating at least two bank account objects and performing various operations on them.

Assignment: Student Result Management System using C++

Problem Statement

Design a Student class to store student details and marks in three subjects.
The program should calculate total marks, assign grades, and maintain shared information such as the total number of students and the student with the highest total marks.

Requirements

1. Create a Student class with private data members:
    - Student name
    - Roll number
    - Class/Section
    - Marks in three subjects (individual variables)
2. Implement:
    - A constructor to initialize student details and marks
    - A member function to calculate total marks
    - A member function to calculate grade based on total marks
    - A member function to update marks (Note: While updating student marks, the updated marks for **all three subjects must be provided at the same time**.
      This ensures that the **total marks and grade are recalculated correctly)**
    - A member function to display student details, total marks, and grade
3. Use **static data members** to:
    - Store the total number of students
    - Store the highest total marks
    - Store the name of the student with the highest marks
4. Use **static member functions** to:
    - Display total number of students
    - Display the highest scorer details (name and marks)
5. Demonstrate the program by creating multiple student (at least 3) objects and performing updates.

Assignment: Demonstration of Compile-Time Polymorphism using Function Overloading

Problem Statement

Design a C++ program to demonstrate **function overloading** by creating a class that performs addition operations on different data types and different numbers of operands. The same function name should be used to add integers, floating-point numbers, double values, and strings.

Requirements

1. Create a class named AddDemo.
2. Implement overloaded add() functions to:
    - Add **two integers**
    - Add **three integers**
    - Add **two floats**
    - Add **three floats**
    - Add **two doubles**
    - Add **three doubles**
    - Concatenate **two strings**
3. Ensure that:
    - All functions use the **same function name**
    - The functions differ only in **number and/or type of parameters**
4. Demonstrate the working of each overloaded function in the main() function.

Assignment: Operator Overloading in C++

Problem Statement

Design a C++ program to represent a 2-D point using a Class named Point.
Each Point object should store integer coordinates x and y.

The program should demonstrate **operator overloading** by performing arithmetic, comparison, and increment/decrement operations on Point objects.

Requirements

1. Create a Class named Point with the following:
    - Two integer data members: x and y
    - A constructor with default values for initialization
2. Overload the following operators for the Point structure:

    Arithmetic Operators

    - + : Add two Point objects
      (x coordinates add together, y coordinates add together)
    - - : Subtract one Point object from another

    Comparison Operator

    - == : Compare two Point objects for equality
      (Both x and y values must be equal)

    Increment / Decrement Operators

    - ++ (Pre-increment): Increase both x and y by 1
    - ++ (Post-increment): Return the original value, then increment
    - -- (Pre-decrement): Decrease both x and y by 1
    - -- (Post-decrement): Return the original value, then decrement
3. Implement a display () function to print the point in (x, y) format.
4. Demonstrate the working of all overloaded operators in the main() function using multiple Point objects.

Important Notes

- Pre-increment and pre-decrement operators should return a **reference** to the current object.
- Post-increment and post-decrement operators should return a **copy** of the original object.
- Use appropriate comments to explain how each operator works.

Inheritance Code Challenges

1. Single Inheritance – Banking System

Problem:

Create a base class Account with:

- accountNumber, balance
- Methods: deposit(), withdraw()

Create a derived class SavingsAccount with:

- interestRate
- A method applyInterest() to add interest to the balance

Challenge Goals:

- Use protected members
- Show base method reuse
- Try calling applyInterest() and verify output

Assignment: Multilevel Inheritance – Smart Appliances

Problem Statement

Design a set of classes representing smart home appliances. The hierarchy is as follows:

Appliance → SmartAppliance → SmartFridge

Class Requirements

1. **Appliance (Base Class)**
    - Data Members: string brand; bool isOn;
    - Constructor: Set brand, initialize isOn = false.
    - Method:
        - void powerOn() → Set isOn = true and print "Appliance [brand] is powered on."
2. **SmartAppliance (Derived from Appliance)**
    - Data Members: string wifiNetwork; bool connected;
    - Constructor: Initialize brand via parent class, set wifiNetwork, connected = false.
    - Method:
        - void connectToWiFi() → If isOn is true, set connected = true and print "SmartAppliance [brand] connected to [wifiNetwork]."
3. **SmartFridge (Derived from SmartAppliance)**
    - Data Members: int temperature;
    - Constructor: Initialize all parent data members and temperature.
    - Methods:
        - void autoDefrost() → Print "SmartFridge [brand] is auto-defrosting."
        - void trackInventory() → Print "SmartFridge [brand] inventory tracking enabled."

Assignment Tasks

1. Create **objects** of SmartFridge.
2. Call the following methods in sequence from the SmartFridge object:
    - powerOn()
    - connectToWiFi()
    - autoDefrost()
    - trackInventory()
3. Show that all parent methods are accessible from the most derived class.

4. Display clear output demonstrating the sequence of operations.

Sample Input/Output

Input (user-defined if needed):

Brand: CoolTech

WiFi Network: HomeWiFi

Temperature: 4

Output:

Appliance CoolTech is powered on.

SmartAppliance CoolTech connected to HomeWiFi.

SmartFridge CoolTech is auto-defrosting.

SmartFridge CoolTech inventory tracking enabled.

Optional Challenges

1. Add **error handling**: Cannot connect to WiFi if the appliance is off.
2. Add **temperature control**: Set and display current fridge temperature.
3. Implement **constructor chaining** with initialization lists.

Hints for Students

- Use : public ParentClass syntax for inheritance.
- Call parent constructors using **initialization lists** in derived classes.
- Pay attention to the **order of method calls** to respect appliance logic.
- Use proper **encapsulation** (private/protected members)

Assignment: Multiple Inheritance – Event Management System

Problem Statement

Design a **conference management system**. The system involves multiple inheritance:

VenueDetails EventDetails

\ /

\ /

Conference

Class Requirements

1. **VenueDetails (Base Class)**

   Data Members:

   - string venueName;
   - int capacity;

   Constructor: Initialize venueName and capacity.

   Method:

   - void displayVenueInfo(); // Prints: "Venue: [venueName], Capacity: [capacity]"

2. **EventDetails (Base Class)**

   Data Members:

   - string title;
   - string date;

   Constructor: Initialize title and date.

   Method:

   - void displayEventInfo(); // Prints: "Event: [title], Date: [date]"

3. **Conference (Derived Class, inherits from VenueDetails and EventDetails)**

   Data Members:

   - string speakerList[5]; // Maximum 5 speakers
   - int speakerCount; // Number of speakers added

   Constructor: Initialize parent data members and set speakerCount = 0.

   Methods:

   - void addSpeaker(string name); // Adds a speaker to the list

- void displayConferenceInfo(); // Displays venue, event, and speakers

Assignment Tasks

1. Create at least **one Conference object**.
2. Populate the conference with:
    - Venue details
    - Event details
    - Up to 5 speakers using addSpeaker()
3. Display the full conference information using displayConferenceInfo().
4. Demonstrate that all parent methods can still be accessed if needed (e.g., displayVenueInfo()).

Sample Input/Output

Input (user-defined or hardcoded):

Venue: Grand Hall

Capacity: 250

Event: Tech Conference

Date: 2025-12-30

Speakers: Alice, Bob, Charlie

Output:

Conference Details:

Venue: Grand Hall, Capacity: 250

Event: Tech Conference, Date: 2025-12-30

Speakers:

1. Alice

2. Bob

3. Charlie

Optional Challenges

1. Allow **adding/removing speakers** dynamically with proper error handling.
2. Implement **constructor chaining** to initialize both parent classes from the derived class.
3. Add **validation**: Capacity must be positive; date format should be valid.

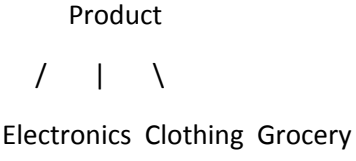
Hints for Students

- Use the syntax : public Parent1, public Parent2 for multiple inheritance.
- Call parent constructors using **initialization lists** in the derived class.
- Access parent methods as usual; use scope resolution if there's any ambiguity.
- Use a simple for loop to iterate over the speakerList array for display.
- Keep speakerCount updated when adding new speakers.

Assignment: Hierarchical Inheritance – Online Shopping System

Problem Statement

Design an **online shopping system**. The system has one base class Product and three derived classes representing different product categories:



Class Requirements

1. **Product (Base Class)**

   Data Members:

   - string productID;
   - string name;
   - double price;

   Constructor: Initialize all data members.

   Method:

   - void displayProductInfo(); // Prints: "ID: [productID], Name: [name], Price: [price]"

2. **Electronics (Derived Class)**

   Data Members:

   - int warranty; // Warranty in months

   Constructor: Initialize base class members and warranty.

   Method:

   - void displayElectronicsInfo(); // Prints product info + warranty

3. **Clothing (Derived Class)**

   Data Members:

   - string size;
   - string material;

   Constructor: Initialize base class members and size & material.

   Method:

   - void displayClothingInfo(); // Prints product info + size and material

4. **Grocery (Derived Class)**

   Data Members:

- string expiryDate; // Format: YYYY-MM-DD

Constructor: Initialize base class members and expiryDate.

Method:

- void displayGroceryInfo(); // Prints product info + expiry date

Assignment Tasks

1. Create at least **one object** of each derived class: Electronics, Clothing, and Grocery.
2. Initialize each object with appropriate details.
3. Call the corresponding **display method** for each object to show all information.
4. Demonstrate that **base class methods** are accessible from derived class objects.

Sample Input/Output

Input (user-defined or hardcoded):

Electronics: ID E101, Name: Smartphone, Price: 35000, Warranty: 24 months

Clothing: ID C202, Name: T-Shirt, Price: 799, Size: M, Material: Cotton

Grocery: ID G303, Name: Milk, Price: 60, Expiry: 2025-12-30

Output:

Electronics Product:

ID: E101, Name: Smartphone, Price: 35000

Warranty: 24 months

Clothing Product:

ID: C202, Name: T-Shirt, Price: 799

Size: M, Material: Cotton

Grocery Product:

ID: G303, Name: Milk, Price: 60

Expiry Date: 2025-12-30

Optional Challenges

1. Add **discount calculation** for each product type and display the discounted price.
2. Add **input validation**: Price must be positive, warranty non-negative, expiry date valid.

Hybrid Inheritance – University Structure

Problem Statement

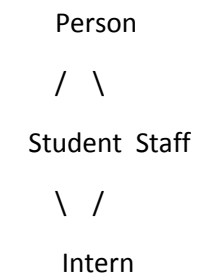
A university system maintains information about **people** associated with the institution.
A person can be a **student**, a **staff member**, or an **intern**.

An **Intern** is both:

- A **Student** (has a roll number)
- A **Staff member** (has an employee ID)

This creates a **diamond inheritance structure**, which must be handled carefully to avoid duplicate base class objects.

Inheritance Structure



Class Design Requirements

Base Class: Person

Data Members:

- name
- email

Responsibilities:

- Store common personal details
- Provide a function to display personal information

Derived Class: Student

Inheritance Type:

- **Virtually inherit** from Person

Data Members:

- rollNumber

Responsibilities:

- Initialize student-specific data

- Display student details

Derived Class: Staff

Inheritance Type:

- **Virtually inherit** from Person

Data Members:

- employeeId

Responsibilities:

- Initialize staff-specific data
- Display staff details

Derived Class: Intern

Inheritance Type:

- Inherit from both Student and Staff

Data Members:

- assignedProject

Responsibilities:

- Initialize all inherited data members
- Display complete intern details
- Access Person, Student, and Staff members without ambiguity

Constructor Requirements

- Each class must have a **parameterized constructor**
- The Intern constructor must:
    - Initialize Person, Student, and Staff
    - Demonstrate correct constructor initialization order
- Avoid duplicate initialization of Person

Main Function Requirements

In main():

1. Create an Intern object with:
    - Name

- Email
- Roll number
- Employee ID
- Assigned project

2. Call a function to display:
    - Personal details
    - Student details
    - Staff details
    - Intern project details
3. Ensure there is:
    - No ambiguity
    - No compiler errors
    - Clear, correct output

Expected Output (Sample)

Name: Ananya Sharma

Email: ananya@university.edu

Roll Number: ST202

Employee ID: EMP451

Assigned Project: AI Research System

LAB ASSIGNMENT: Polymorphic Billing System

Assignment Objective

To design and implement a **polymorphic billing system** using **inheritance and runtime polymorphism**, where different customer types calculate their bills differently but are processed uniformly using **base class pointers**.

Problem Statement

An electricity distribution company serves **different types of customers**, each with **different billing rules**.
The company wants a system where all customers can be handled **polymorphically**, without knowing their exact type at compile time.

You are required to design a billing system that:

- Calculates bills based on customer type
- Applies **discounts for high unit consumption**
- Computes the **total revenue collected from all customers**

Class Design Requirements

Base Class: Customer

Data Members:

- customerId
- customerName
- unitsConsumed

Member Functions:

- virtual double calculateBill()
- virtual void displayBill()
- Virtual destructor

The calculateBill() function must be declared virtual to enable runtime polymorphism.

Derived Class: ResidentialCustomer

Billing Rules:

- 5 per unit for all units consumed

Discount Rule:

- If units consumed **exceed 300 units**, apply a **10% discount** on the total bill

Responsibilities:

- Override calculateBill()
- Override displayBill()
- Use the override keyword

Derived Class: CommercialCustomer

Billing Rules:

- 8 per unit
- Additional fixed service charge of 200

Discount Rule:

- If units consumed **exceed 500 units**, apply a **5% discount** on the total bill

Responsibilities:

- Override calculateBill()
- Override displayBill()
- Use the override keyword

Derived Class: IndustrialCustomer

Billing Rules:

- 10 per unit
- Fixed meter charge of 500

Discount Rule:

- If units consumed **exceed 1000 units**, apply a **7% discount** on the total bill

Responsibilities:

- Override calculateBill()
- Override displayBill()
- Use the override keyword

Runtime Polymorphism Requirements (VERY IMPORTANT)

- Use **base class pointers (Customer*)** to store derived objects
- Call displayBill() and calculateBill() through base class pointers

- Ensure the **correct derived version** executes at runtime

Do NOT:

- Store derived objects by value in base class variables
- Pass objects by value to functions

(This avoids the **object slicing problem**.)

Total Revenue Requirement

Implement a **separate function (NOT BELONGING TO ANY CLASS)** that:

- Accepts an array of Customer*
- Iterates through all customers
- Uses calculateBill() polymorphically
- Computes and returns the **total revenue collected**

Example:

double calculateTotalRevenue(Customer* customers[], int count);

Main Function Requirements

In main():

1. Create at least:
    - One ResidentialCustomer
    - One CommercialCustomer
    - One IndustrialCustomer
2. Store them in:
3. Customer* customers[3];
4. Use a loop to:
    - Display individual bills
    - Calculate and display total revenue

Expected Output (Sample)

Customer Type: Residential

Customer ID: 101

Units Consumed: 350

Total Bill: 1575

Customer Type: Commercial

Customer ID: 102

Units Consumed: 600

Total Bill: 4940

Customer Type: Industrial

Customer ID: 103

Units Consumed: 1200

Total Bill: 11346

Total Revenue Collected: 17861

Design Constraints

- Function signatures must match exactly when overriding
- override keyword must be used in derived classes
- Virtual destructor must be present in base class
- Program must demonstrate runtime polymorphism clearly
- No object slicing should occur

Code Challenge: Abstract Classes & Pure Virtual Functions with Vehicles

Problem Statement:

Design a class hierarchy that represents different types of vehicles using **abstract classes** and **pure virtual functions**.

You are required to:

1. Create an **abstract base class Vehicle** with behaviours common to all vehicles.
2. Derive specific classes like **Car** and **Bike** from Vehicle.
3. Use **pure virtual functions** to enforce implementation of vehicle-specific details.
4. Implement **polymorphism** using base class pointers to manage different vehicle types dynamically.

Class Requirements:

Base Class: Vehicle (Abstract)

- Pure Virtual Methods:
    - void move() → define how the vehicle moves
    - void fuelType() → specify the fuel used
    - void numberOfWheels() → return number of wheels
    - void engineCapacity() → return engine size in cc
- Utility Functions:
    - void stop() → common stop message
    - void calculateMileage(int distance, int fuelUsed) → calculate and display mileage

Derived Class: Car

- Implements all pure virtual methods:
    - Move behavior: highway driving
    - Fuel type: petrol/diesel
    - Wheels: 4
    - Engine: 1500cc

Derived Class: Bike

- Implements all pure virtual methods:
    - Move behavior: traffic navigation
    - Fuel type: petrol
    - Wheels: 2

- Engine: 220cc

Challenge Goals:

Demonstrate **abstract class** usage
Implement **pure virtual functions**
Show **runtime polymorphism** with base class pointers
Add some logical methods like mileage and stop behaviour